

```
$match: {  
  status: "active"  
}
```

```
db.users.createIndex(  
  { email: 1 },  
  { unique: true }  
)
```

MONGODB CHEAT SHEET

THE ULTIMATE DEVELOPER REFERENCE • 2026

✓ Interview Ready

✓ MERN Stack Focus

✓ Production Proven

✓ Aggregation & Atlas

```
_id: ObjectId(  
  "64d3a...f8"  
)
```

```
{ $lookup: {  
  from: "orders",  
  as: "userOrders"  
}}
```

ALGOTUTOR

LEARN SMARTER • BUILD FASTER • CRACK INTERVIEWS

Welcome to MongoDB



What is MongoDB?

MongoDB is a source-available cross-platform document-oriented database program. Classified as a NoSQL database program, MongoDB uses JSON-like documents with optional schemas.

Why developers love it:

- Flexible Schema Design
- Scalability (Horizontal scaling via Sharding)
- Performance & Speed
- Rich Query Language



Who should learn MongoDB?

- **MERN/MEAN Stack Devs:** It's the 'M' in the stack.
- **Backend Engineers:** For handling unstructured data.
- **Data Architects:** For scalable systems.



Career & Market (2026)

Top Companies Using MongoDB: Google, Stripe, Apple, Uber, Coinbase, SEGA, EA.

Average Salaries (Global/Tier 1):

Junior Dev: \$75k - \$100k

Senior MERN Dev: \$130k - \$180k

Database Architect: \$160k+



Pro Tip

MongoDB is perfect for catalogs, content management, real-time analytics, and applications requiring rapid iterative development.

1. MongoDB Fundamentals

Core Concepts

- **NoSQL:** Not Only SQL. Non-relational, flexible schema.
- **Document Database:** Stores data in documents (BSON).
- **JSON/BSON:** JSON (JavaScript Object Notation) is used to view data, but MongoDB stores it in BSON (Binary JSON) for efficiency and support for more data types (Date, ObjectId).
- **Database:** A container for collections.
- **Collection:** A grouping of MongoDB documents (like a SQL Table).
- **Document:** A single record (like a SQL Row).
- **Field:** A key-value pair in a document (like a SQL Column).

Architecture Basics

- **_id (ObjectId):** Every document requires a unique primary key. If not provided, MongoDB auto-generates an ObjectId (12-byte hex string).
- **Replica Sets:** A group of MongoDB instances that maintain the same data set. Provides redundancy and high availability.
- **Sharding:** Distributing data across multiple machines to support massive data sets and high throughput operations.
- **MongoDB Atlas:** The fully managed global cloud database service by MongoDB.

2. Installation & Ecosystem

Component	Description	Use Case
MongoDB Community	Free, self-managed local database.	Local development & testing.
MongoDB Atlas	Managed cloud service (AWS/GCP/Azure).	Production, global scale, auto-backups.
MongoDB Compass	Official GUI for MongoDB.	Visualizing data, writing aggregation pipelines.
Mongosh	Modern interactive JavaScript shell.	CLI administration, testing queries quickly.
Mongoose / Drivers	Node.js ODM / Language specific drivers.	Application code integration.

3. Essential Shell Commands

Database & Collection Management

```
// List all databases
show dbs

// Switch to or create a database
use myDatabase

// Show current database
db

// List collections in current database
show collections

// Create a collection explicitly (optional)
db.createCollection("users")

// Drop a collection
db.users.drop()

// Drop the current database
db.dropDatabase()

// Clear console screen
cls

// Check server stats
db.serverStatus()
```

4. CRUD Cheat Sheet

Create (Insert)

```
// Insert a single document
db.users.insertOne({
  name: "Alice",
  age: 28,
  status: "active"
})

// Insert multiple documents
db.users.insertMany([
  { name: "Bob", age: 35 },
  { name: "Charlie", age: 22 }
])
```

Read (Find)

```
// Find all documents
db.users.find()

// Find with filter
db.users.find({ age: { $gt: 25 } })

// Find one document
db.users.findOne({ name: "Bob" })

// Query with chaining (Sort, Skip, Limit)
db.users.find()
  .sort({ age: -1 }) // -1 Desc, 1 Asc
  .skip(10)          // Pagination
  .limit(5)          // Returns 5 docs

// Count documents
db.users.countDocuments({ status:
"active" })
```

Update

```
// Update the first matching document
db.users.updateOne(
  { name: "Alice" }, // Filter
  { $set: { age: 29 } } // Update
)

// Update all matching documents
db.users.updateMany(
  { status: "active" },
  { $set: { verified: true } }
)

// Upsert (Insert if not found)
db.users.updateOne(
  { name: "Dave" },
  { $set: { age: 40 } },
  { upsert: true }
)
```

Delete

```
// Delete one document
db.users.deleteOne({ name:
"Charlie" })

// Delete all matching documents
db.users.deleteMany({ status:
"inactive" })

// Delete ALL documents in collection
db.users.deleteMany({})
```

Interview Note

Know the difference between `updateOne()` and `replaceOne()`. `updateOne()` modifies specific fields (using `$set`). `replaceOne()` overwrites the entire document except the `_id`.

5. Query Operators

Type	Operator	Description	Example	
Comparison	<code>\$eq</code> / <code>\$ne</code>	Equal / Not Equal	<code>{ age: { \$ne: 30 } }</code>	
	<code>\$gt</code> / <code>\$gte</code>	Greater Than / Greater or Equal	<code>{ price: { \$gte: 100 } }</code>	
	<code>\$lt</code> / <code>\$lte</code>	Less Than / Less or Equal	<code>{ score: { \$lt: 50 } }</code>	
	<code>\$in</code>	Matches any value in an array	<code>{ status: { \$in: ["A", "B"] } }</code>	
	<code>\$nin</code>	Matches none of the values	<code>{ category: { \$nin: ["spam"] } }</code>	
	Logical	<code>\$and</code>	Logical AND	<code>{ \$and: [{ qty: { \$lt: 10 } }, { qty: { \$gt: 5 } }] }</code>
		<code>\$or</code>	Logical OR	<code>{ \$or: [{ status: "A" }, { qty: { \$lt: 30 } }] }</code>
<code>\$not</code>		Inverts the effect of a query expression	<code>{ price: { \$not: { \$gt: 1.99 } } }</code>	
Element	<code>\$exists</code>	Matches documents that have the field	<code>{ qty: { \$exists: true, \$nin: [5, 15] } }</code>	
	<code>\$type</code>	Selects docs if field is of specified type	<code>{ zipCode: { \$type: "string" } }</code>	
Array	<code>\$all</code>	Matches arrays that contain all elements	<code>{ tags: { \$all: ["ssl", "security"] } }</code>	
	<code>\$size</code>	Matches arrays of a specified size	<code>{ comments: { \$size: 3 } }</code>	
	<code>\$elemMatch</code>	Matches array element satisfying all conditions	<code>{ results: { \$elemMatch: { \$gte: 80, \$lt: 85 } } }</code>	

6. Projection Operators

Projection limits the fields returned in a query. `1` includes, `0` excludes.

```
// Return only name and age, exclude _id
db.users.find(
  { status: "A" },
  { name: 1, age: 1, _id: 0 }
)

// Projection on nested documents
db.users.find(
  {},
  { "address.city": 1 }
)
```

⚠ Warning

You cannot mix inclusion (1) and exclusion (0) in a single projection, EXCEPT when excluding the `_id` field.

7. Update Operators

- `$set` : Sets the value of a field.
- `$unset` : Removes the specified field.
- `$inc` : Increments the value of a field.
- `$mul` : Multiplies the value of a field.
- `$rename` : Renames a field.
- `$push` : Appends a value to an array.
- `$pull` : Removes elements from an array.
- `$addToSet` : Adds unique value to an array.

```
// Example
db.users.updateOne(
  { _id: 1 },
  {
    $set: { updatedAt: new Date() },
    $inc: { loginCount: 1 },
    $push: { roles: "admin" }
  }
)
```

8. Aggregation Framework

The aggregation pipeline processes data records and returns computed results. Documents pass through stages sequentially.

Core Pipeline Stages

\$match	Filters documents (like <code>find</code> or SQL <code>WHERE</code>). Should be placed early to reduce pipeline data.
\$group	Groups documents by a specified <code>_id</code> expression and outputs a document for each distinct grouping (SQL <code>GROUP BY</code>).
\$project	Reshapes each document in the stream, like adding new fields or removing existing ones (SQL <code>SELECT</code>).
\$sort	Reorders the document stream by a specified sort key (SQL <code>ORDER BY</code>).
\$limit / \$skip	Restricts the number of documents / skips documents.
\$lookup	Performs a left outer join to an unsharded collection in the same database (SQL <code>JOIN</code>).
\$unwind	Deconstructs an array field from the input documents to output a document for each element.

Example: Total Sales by Item

```
// Calculates total revenue per item, sorted by highest revenue
db.orders.aggregate([
  { $match: { status: "completed" } },
  {
    $group: {
      _id: "$item",
      totalRevenue: { $sum: { $multiply: [ "$price", "$quantity" ] } }
    }
  },
  { $sort: { totalRevenue: -1 } },
  { $limit: 5 }
])
```


Example: \$lookup (Joins in MongoDB)

```
db.orders.aggregate([
  {
    $lookup: {
      from: "inventory",           // Collection to join
      localField: "item",          // Field from the input documents
      foreignField: "sku",         // Field from the documents of the "from" collection
      as: "inventory_docs"        // Output array field
    }
  }
])
```

9. Indexing

Indexes support the efficient execution of queries. Without indexes, MongoDB performs a *collection scan* (reads every document).

- **Single Field:** `db.col.createIndex({ age: 1 })`
- **Compound:** `db.col.createIndex({ age: 1, name: -1 })`
- **Text Index:** `db.col.createIndex({ comments: "text" })`
- **Unique Index:** `db.col.createIndex({ email: 1 }, { unique: true })`
- **TTL (Time-To-Live):** Automatically delete documents after a certain time. Useful for sessions/logs.
`db.col.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })`

💡 ESR Rule for Compound Indexes

Equality, Sort, Range. Fields matching exactly should come first, then fields used for sorting, and finally range filters (\$gt, \$lt).

10. Data Modeling

In MongoDB, data modeling is driven by application access patterns, not by data normalization.

Embedding (Denormalization)

- Use for "Contains" relationships (One-to-Few).
- Data read/updated together.
- Fewer queries, faster reads.
- *Warning:* Document size limit is 16MB.

Referencing (Normalization)

- Use for "One-to-Many" or "Many-to-Many".
- Store ObjectIds and use `$lookup` or `.populate()`.
- Provides flexibility and smaller documents.
- *Trade-off:* Requires multiple queries/joins.

12 & 13. Mongoose & Node.js

Mongoose Setup & Schema

```
const mongoose = require('mongoose');

// Connect to Database
mongoose.connect(process.env.MONGO_URI);

// Define Schema
const userSchema = new
mongoose.Schema({
  name: { type: String, required:
true },
  email: { type: String, unique:
true },
  age: { type: Number, min: 18 },
  role: { type: String, enum: ['user',
'admin'] }
}, { timestamps: true }); // Adds
createdAt & updatedAt

// Create Model
const User = mongoose.model('User',
userSchema);
```

Mongoose Middleware & Virtuals

```
// Pre-save Hook (Middleware) - Hash
password
userSchema.pre('save', async function(next)
{
  if (!this.isModified('password'))
return next();
  this.password = await
bcrypt.hash(this.password, 10);
  next();
});

// Virtual Property (Not saved to DB)
userSchema.virtual('domain').get(function()
{
  return this.email.split('@')[1];
});
```

14. Performance Optimization

15. Security Best Practices

- **Use Indexes:** Ensure queries hit an index. Use `.explain("executionStats")` to analyze queries.
 - **Limit Results:** Always use `.limit()` for pagination.
 - **Projection:** Only return necessary fields to save network bandwidth and memory.
 - **Connection Pooling:** Do not open/close connections per request. Reuse the db connection object in Node.js.
 - **Bulk Operations:** Use `bulkWrite()` for thousands of inserts/updates instead of a loop.
- **Authentication:** Always enable access control (SCRAM/x.509).
 - **RBAC:** Use Role-Based Access Control. Assign least privilege users.
 - **Network Isolation:** IP Whitelisting in Atlas or bind to localhost/VPC locally.
 - **Encryption:** Use TLS/SSL for data in transit. Enable Encryption at Rest.
 - **Avoid Injection:** In Node.js, sanitize user input to avoid NoSQL injection (e.g., using `mongo-sanitize`).

17. SQL vs MongoDB

SQL Concept	MongoDB Concept	SQL Query	MongoDB Equivalent
Table	Collection	<code>CREATE TABLE users</code>	<code>db.createCollection("users")</code>
Row	Document	<code>INSERT INTO users</code>	<code>db.users.insertOne()</code>
Column	Field	<code>SELECT name FROM users</code>	<code>db.users.find({}, {name: 1})</code>
JOIN	\$lookup	<code>SELECT * FROM a JOIN b</code>	<code>db.a.aggregate([{\$lookup: ...}])</code>
WHERE	Filter Document	<code>WHERE age > 18</code>	<code>{ age: { \$gt: 18 } }</code>
GROUP BY	\$group	<code>GROUP BY status</code>	<code>{ \$group: { _id: "\$status" } }</code>

18. Interview Cheat Sheet

Q: What is a Replica Set?

A group of mongod processes that maintain the same data set. Primary receives writes, Secondaries replicate. Provides High Availability (HA).

Q: How does Sharding work?

Distributes data across multiple clusters. A `shard key` determines data distribution.

Q: CAP Theorem & MongoDB?

MongoDB is a CP system (Consistency & Partition Tolerance). During network partitions, it may sacrifice Availability to maintain Consistency.

Q: What is Write Concern?

Level of acknowledgment requested from MongoDB for write operations (e.g., `w: "majority"`).

20. Common Errors & Fixes

- **MongoServerError: E11000 duplicate key**

error: Tried to insert a document with an existing unique value (often `\_id` or unique index like email).

Fix: Check data or generate unique ID.

- **CastError (Mongoose):** Value provided doesn't match Schema type (e.g., passing a string to an ObjectId field).

Fix: Convert types before querying.

- **Authentication failed:** Wrong username/password or auth DB.

Fix: Check connection string `mongodb+srv://user:pass@cluster...`

- **MongoNetworkError:** IP not whitelisted.

Fix: Add current IP to Atlas Network Access.

"Great developers aren't born—they're built through consistent learning, curiosity, and practice."

ALGOTUTOR

Learn Smarter • Build Faster • Crack Interviews

WWW.ALGOTUTOR.IN